

Notatki z wykładu: Algorytmy zachłanne

Szeregowanie zadań z karami oraz algorytmy aproksymacyjne

29 kwietnia 2026

Część I — Algorytmy zachłanne (kontynuacja)

1 Problem szeregowania zadań

Dany jest zbiór zadań o jednostkowym czasie wykonania

$$S = \{z_1, z_2, \dots, z_n\}.$$

Każde zadanie z_i ma **termin** (deadline) d_i oraz **karę** za opóźnienie w_i :

$$\forall i \in [n] \ d_i \in [n], \quad \forall i \in [n] \ w_i \in \mathbb{R}^+.$$

Karę w_i płacimy za zadanie z_i wtedy, gdy *nie* wyrobimy się z jego wykonaniem do chwili d_i .

Harmonogram to ustalona kolejność wykonywania zadań. Zadanie nazywamy **terminowym**, jeśli kończy się przed swoim terminem; w przeciwnym przypadku jest **spóźnione**.

Celem jest znalezienie harmonogramu minimalizującego łączną karę.

2 Zbiory wolne od kar

Definicja 1 (zbiór wolny od kar). Zbiór zadań $A \subseteq S$ nazywamy **wolnym od kar**, jeśli istnieje harmonogram, w którym wszystkie zadania z A są terminowe.

Dla $t \in \{0, 1, \dots, n\}$ wprowadzamy oznaczenie

$$N_t(A) = |\{z_i \in A : d_i \leq t\}| \quad (\text{liczba zadań z } A \text{ o terminie nie późniejszym niż } t).$$

Lemat 1 (charakteryzacja zbiorów wolnych od kar). *Dla dowolnego zbioru $A \subseteq S$ następujące warunki są równoważne:*

- (1) A jest wolny od kar;
- (2) $N_t(A) \leq t$ dla każdego $t \in \{1, \dots, n\}$;
- (3) jeśli zadania z A ustawimy w kolejności niemalejącej według terminów, to żadne zadanie nie jest spóźnione.

Dowód. **(1) $\Rightarrow$ (2).** Przypuśćmy nie wprost, że istnieje t takie, że $N_t(A) > t$. Wtedy w A jest więcej niż t zadań o terminie $\leq t$. Przy jednostkowym czasie wykonania do chwili t można ukończyć co najwyżej t zadań, więc któreś z tych zadań musi być spóźnione — sprzeczność z tym, że A jest wolny od kar. Zatem $N_t(A) \leq t$ dla każdego t .

(2) $\Rightarrow$ (3). Ustawmy zadania niemalejąco według terminów i przypuśćmy nie wprost, że pewne zadanie z kończy się w chwili $\ell > d_z$ (jest spóźnione). Ponieważ kolejność jest niemalejąca, wszystkie ℓ zadań wykonanych do chwili ℓ ma termin $\leq d_z$. Stąd

$$N_{d_z}(A) \geq \ell > d_z,$$

co przeczy warunkowi (2). Zatem żadne zadanie nie jest spóźnione.

(3) $\Rightarrow$ (1). Oczywiście: harmonogram posortowany według terminów jest harmonogramem bez spóźnień, więc A jest wolny od kar. $\square$

2.1 Sprowadzenie do problemu zachłannego

Chcemy znaleźć zbiór A wolny od kar taki, że suma kar zadań spóźnionych

$$\sum_{z_i \notin A} w_i$$

jest najmniejsza możliwa. Ponieważ

$$\sum_{z_i \notin A} w_i = \sum_{z_i \in S} w_i - \underbrace{\sum_{z_i \in A} w_i}_{(*)}$$

a pierwszy składnik jest stały, minimalizacja kary jest *równoważna* maksymalizacji sumy $(*)$ wag zadań terminowych. Szukamy więc wolnego od kar zbioru A maksymalizującego $\sum_{z_i \in A} w_i$.

3 Struktura matroidu

Niech $\mathcal{A}$ będzie rodziną wszystkich zbiorów wolnych od kar.

Twierdzenie 1. Niech S będzie zbiorem zadań o jednostkowym czasie wykonania, a $\mathcal{A}$ rodziną zbiorów wolnych od kar. Wtedy $(S, \mathcal{A})$ jest **matroidem**.

Dowód. Sprawdzamy cztery warunki definicji matroidu.

(1) **Skończoność.** Zbiór S jest skończony — oczywiście.

(2) **Niepustość.** $\mathcal{A}$ jest niepustym podzbiorem zbioru potęgowego 2^S : dla każdego $i \in [n]$ pojedyncze zadanie $\{z_i\}$ jest wolne od kar (można je wykonać do chwili $d_i \geq 1$), więc $\{z_i\} \in \mathcal{A}$.

(3) **Dziedziczność.** Niech $A \in \mathcal{A}$ oraz $B \subseteq A$. Istnieje harmonogram, w którym wszystkie zadania z A są terminowe. Bierzemy ten harmonogram i usuwamy z niego wszystkie zadania nienależące do B ; otrzymujemy harmonogram dla B , w którym zadania nadal pozostają terminowe. Zatem $B \in \mathcal{A}$.

(4) **Własność wymiany.** Niech $A, B \in \mathcal{A}$ oraz $|B| > |A|$. Zdefiniujemy

$$k = \text{największa liczba } t \in \{0, 1, \dots, n\} \text{ taka, że } N_t(B) \leq N_t(A).$$

Liczba k jest dobrze określona, bo $N_0(B) = 0 = N_0(A)$, więc warunek zachodzi dla $t = 0$. Podstawiając $t = n$ dostajemy

$$N_n(B) = |B| > |A| = N_n(A),$$

więc warunek $N_t(B) \leq N_t(A)$ nie zachodzi dla $t = n$, skąd $k < n$. Z definicji k mamy ponadto

$$N_k(B) \leq N_k(A) \quad \text{oraz} \quad N_{k+1}(B) > N_{k+1}(A).$$

Stąd istnieje co najmniej jedno zadanie $z \in B \setminus A$ o terminie równym $k + 1$. Połóżmy

$$A' = A \cup \{z\}.$$

Pokażemy, używając warunku (2) Lematu 1, że A' jest wolny od kar:

- dla $t \in \{1, \dots, k\}$: zadanie z ma termin $k + 1 > t$, więc nie jest liczone, i

$$N_t(A') = N_t(A) \leq t \quad (\text{bo } A \text{ jest wolny od kar});$$

- dla $t \in \{k + 1, \dots, n\}$: z definicji k mamy $N_t(A) < N_t(B)$, czyli $N_t(A) + 1 \leq N_t(B)$, zatem

$$N_t(A') = N_t(A) + 1 \leq N_t(B) \leq t \quad (\text{bo } B \text{ jest wolny od kar}).$$

W obu przypadkach $N_t(A') \leq t$, więc A' jest wolny od kar, czyli $A \cup \{z\} \in \mathcal{A}$. To kończy dowód własności wymiany, a tym samym dowód, że $(S, \mathcal{A})$ jest matroidem. $\square$

Wniosek 1 (twierdzenie Rado–Edmondsa). *Ponieważ $(S, \mathcal{A})$ jest matroidem, algorytm zachłanny zastosowany do tego problemu (z wagami w_i jako kluczem) zwraca rozwiązanie optymalne.*

Algorytm. Sortujemy zadania nierosnąco według kar w_i i kolejno próbujemy dodać każde zadanie do budowanego zbioru A , zachowując je wtedy i tylko wtedy, gdy A pozostaje wolny od kar.

Złożoność. Sortowanie kosztuje $O(n \log n)$, a sprawdzanie warunku „wolny od kar” dla kolejnych zadań — $O(n)$ na zadanie, czyli $O(n^2)$ łącznie. Stąd

$$O(n \log n + n \cdot n) = O(n^2).$$

4 Przykład

Rozważmy $n = 8$ zadań o następujących terminach i karach:¹

	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8
d_i	3	2	2	3	1	4	6	8
w_i	80	70	60	50	40	30	20	10

Przetwarzamy zadania w kolejności nierosnących kar (czyli $z_1, z_2, \dots, z_8$) i dla każdego sprawdzamy, czy po dodaniu zbiór pozostaje wolny od kar (warunek $N_t \leq t$):

¹W oryginalnych notatkach termin d_3 był nieczytelny; przyjęto $d_3 = 2$, co jest spójne z końcowym harmonogramem zapisanym w notatkach. Warto to zweryfikować.

krok	rozważane zadanie	bieżący zbiór A	decyzja
1	z_1 ($d=3$)	$\{z_1\}$	przyjmij
2	z_2 ($d=2$)	$\{z_1, z_2\}$	przyjmij
3	z_3 ($d=2$)	$\{z_1, z_2, z_3\}$	przyjmij
4	z_4 ($d=3$)	—	odrzuć (N_3 wzrósłby do $4 > 3$)
5	z_5 ($d=1$)	—	odrzuć (N_2 wzrósłby do $3 > 2$)
6	z_6 ($d=4$)	$\{z_1, z_2, z_3, z_6\}$	przyjmij
7	z_7 ($d=6$)	$\{z_1, z_2, z_3, z_6, z_7\}$	przyjmij
8	z_8 ($d=8$)	$\{z_1, z_2, z_3, z_6, z_7, z_8\}$	przyjmij

Wynikowy zbiór terminowy to $A = \{z_1, z_2, z_3, z_6, z_7, z_8\}$, a zadaniami spóźnionymi są z_4 i z_5 . Suma kar wynosi

$$w_4 + w_5 = 50 + 40 = 90.$$

Ostateczny harmonogram (zadania terminowe według terminów, a na końcu spóźnione):

$$(z_2, z_3, z_1, z_6, z_7, z_8 \mid z_4, z_5).$$

Część II — Algorytmy aproksymacyjne

5 Algorytmy aproksymacyjne

Problemy, dla których znane są algorytmy wielomianowe, nazywamy *łatwymi*. Tam, gdzie takie algorytmy nie są znane, mówimy o problemach *trudnych*. Dla tych ostatnich **algorytm aproksymacyjny** znajduje rozwiązanie *prawie* optymalne.

Niech Π będzie problemem optymalizacyjnym (maksymalizacji albo minimalizacji pewnej funkcji celu f). Wprowadzamy oznaczenia:

- D_Π — zbiór instancji problemu Π , a $I \in D_\Pi$ — pojedyncza instancja;
- $S(I)$ — zbiór rozwiązań dopuszczalnych instancji I ;
- $f : \bigcup_{I \in D_\Pi} S(I) \rightarrow \mathbb{R}$ — funkcja celu;
- $c(I) > 0$ — wartość rozwiązania zwróconego przez algorytm dla instancji I ;
- $c^*(I)$ — wartość rozwiązania optymalnego dla instancji I .

Definicja 2 (współczynnik aproksymacji). Mówimy, że algorytm A ma **współczynnik aproksymacji** $\rho(n)$, jeśli dla każdej instancji $I \in D_\Pi$ rozmiaru n zachodzi

$$\max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \leq \rho(n).$$

Wtedy A nazywamy algorytmem $\rho(n)$ -aproksymacyjnym.

Konwencja jest tak dobrana, że $\rho(n) \geq 1$ niezależnie od typu problemu:

$$\text{maksymalizacja: } 0 < c(I) \leq c^*(I), \quad \text{minimalizacja: } 0 < c^*(I) \leq c(I).$$

W każdym przypadku jeden z dwóch ilorazów jest równy 1, a drugi ≥ 1 , więc ich maksimum jest ≥ 1 . Przypadek $\rho(n) = 1$ oznacza algorytm *dokładny*.

Definiujemy też współczynnik asymptotyczny

$$\rho = \sup_{n \in \mathbb{N}} \rho(n);$$

mówimy wtedy, że współczynnik aproksymacji algorytmu A wynosi ρ , i nazywamy A algorytmem ρ -aproksymacyjnym.

6 Problem pokrycia wierzchołkowego (Vertex Cover)

Niech $G = (V, E)$ będzie grafem.

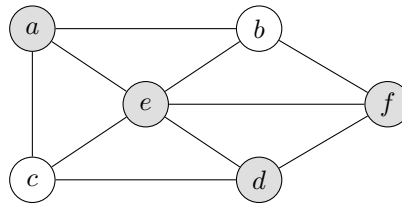
Definicja 3 (pokrycie wierzchołkowe). **Pokryciem wierzchołkowym** grafu G nazywamy zbiór wierzchołków $W \subseteq V$ taki, że każda krawędź ma co najmniej jeden koniec w W :

$$\forall e \in E : e \cap W \neq \emptyset.$$

Problem. Znaleźć pokrycie wierzchołkowe grafu G o *minimalnej* liczbie wierzchołków.

Przykład

W poniższym grafie zbiór $\{a, d, e, f\}$ (wierzchołki zacienione) jest pokryciem wierzchołkowym — każda krawędź ma w nim swój koniec:



Algorytm aproksymacyjny

Algorithm 1 Approx-Vertex-Cover(G)

```

1:  $C \leftarrow \emptyset$  {budowane pokrycie}
2:  $F \leftarrow E(G)$  {zbiór niepokrytych krawędzi}
3: while  $F \neq \emptyset$  do
4:   wybierz dowolną krawędź  $(u, v) \in F$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   usuń z  $F$  wszystkie krawędzie incydentne do  $u$  lub do  $v$ 
7: end while
8: return  $C$ 

```

Przykład działania

Dla grafu z rysunku powyżej kolejne wybory krawędzi mogą dawać:

$$(a, b) \Rightarrow C = \{a, b\}, \quad (c, d) \Rightarrow C = \{a, b, c, d\}, \quad (e, f) \Rightarrow C = \{a, b, c, d, e, f\}.$$

Algorytm zwrócił pokrycie o 6 wierzchołkach (zależnie od wyborów może być mniejsze).

Poprawność

Algorytm zwraca pokrycie wierzchołkowe. W każdym obrocie pętli wybieramy niepokrytą krawędź (u, v) i dodajemy do C oba jej końce, po czym usuwamy z F wszystkie krawędzie incydentne do u lub v . Tym samym w każdym obrocie liczność F maleje o co najmniej 1, a pętla obraca się, dopóki F nie stanie się puste. Zatem każda krawędź zostaje w pewnym momencie pokryta (ma koniec w C), więc C jest pokryciem wierzchołkowym grafu G .